

SMART HOTEL IoT & MONITORING SYSTEM: OTOMATISASI LAYANAN HOUSEKEEPING DAN MANAJEMEN ENERGI BERBASIS EDGE SENSING

Wadu, Luis Henry (24/543995/SV/25305), Nestor Rawikara Sotarduga Hutasoit (24/537963/SV/24456), Muhammad Ilham (24/543413/SV/25208), Muhammad Arvin Kane (24/535212/SV/24174), Ahza Willy Putra (24/534740/SV/24089)

Asisten Praktikum: Arya Bhakti Tri Setia, Haidar Syaouq
Dosen Pengampu: Ir. Unan Yusmaniar Oktiawati, S.T., M.Sc., Ph.D.

nestorrawikarasotardugahutasoit@mail.ugm.ac.id, luishenrywadu@mail.ugm.ac.id, muhammadilham2004@mail.ugm.ac.id,
muhammadarvinkane@mail.ugm.ac.id, ahzawillyputra@mail.ugm.ac.id

Teknologi Rekayasa Internet – Departemen Teknik Elektro dan Informatika
Sekolah Vokasi
Universitas Gadjah Mada

Abstrak—Industri perhotelan modern menghadapi dua tantangan operasional utama yang belum teratasi secara sistematis, yaitu inefisiensi layanan housekeeping akibat fenomena False Make Up Room (MUR) dan pemborosan energi kamar tanpa penghuni. Artikel ini memaparkan perancangan Smart Hotel IoT System berbasis edge sensing yang mengintegrasikan mikrokontroler ESP32 dengan sensor PIR untuk mendeteksi gerak, DHT11 untuk memantau suhu serta kelembapan, dan magnetic switch pemantau status pintu secara real-time. Aktuator relay 5V memutus aliran listrik otomatis ketika kamar terdeteksi kosong. Data dikirimkan melalui protokol MQTT lewat jaringan private Tailscale VPN menuju Ubuntu Server yang menjalankan Mosquitto Broker, InfluxDB sebagai basis data time-series, Apache Spark untuk analitik, dan Grafana sebagai visualisasi dashboard. Metode micro-batching dengan buffer lima detik dan state retention diimplementasikan untuk mengeliminasi nilai data kosong (NaN). Sistem ini mengotomatisasi notifikasi housekeeping, menghindari False MUR, serta menghemat energi lewat kendali relay yang sangat responsif.

Kata kunci—Smart Hotel; Internet of Things; Edge Sensing; Housekeeping Otomatis; Manajemen Energi; ESP32; MQTT; InfluxDB; Grafana; Tailscale VPN.

I. PENDAHULUAN

Transformasi digital yang dipicu oleh era Revolusi Industri 4.0 telah memberikan tekanan yang signifikan pada industri perhotelan untuk beradaptasi dengan paradigma smart environment. Dalam konteks ini, smart environment merujuk pada ekosistem operasional yang mengintegrasikan teknologi informasi dan komunikasi secara menyeluruh ke dalam setiap aspek layanan, mulai dari sistem kendali kamar, pengelolaan energi, otomatisasi layanan tamu, hingga manajemen operasional staf. Tuntutan akan efisiensi yang lebih tinggi, kualitas pengalaman tamu yang lebih baik, serta tekanan biaya operasional yang terus meningkat mendorong para pengelola hotel untuk mencari solusi teknologi yang tidak sekadar berbasis digitalisasi parsial, melainkan integrasi penuh antara perangkat fisik, jaringan komunikasi, dan platform analitik data.

Perkembangan teknologi Internet of Things (IoT), yang merupakan paradigma interkoneksi perangkat fisik melalui

jaringan internet untuk memungkinkan pengumpulan dan pertukaran data secara otomatis, telah membuka peluang besar dalam transformasi operasional hotel. Berbagai penelitian telah membuktikan bahwa penerapan solusi berbasis IoT mampu memperluas kemampuan pengelolaan layanan hotel secara substansial, mulai dari sistem akses kamar berbasis identifikasi digital hingga kerangka manajemen konteks terintegrasi yang meningkatkan pengalaman tamu dan efisiensi operasional secara menyeluruh [1], [6]. Namun demikian, sebagian besar implementasi yang ada masih bersifat parsial, hanya menangani satu aspek operasional tanpa membentuk sebuah pipeline data yang utuh dari lapisan sensor fisik hingga lapisan keputusan manajerial.

Salah satu permasalahan terbesar yang dihadapi oleh operasional housekeeping hotel secara konvensional adalah fenomena yang dikenal sebagai False Make Up Room (MUR). False MUR terjadi ketika tamu secara tidak sengaja atau karena kelupaan menekan tombol permintaan Make Up Room, padahal tamu tersebut masih berada di dalam kamar. Kondisi ini memaksa petugas housekeeping untuk mendatangi kamar dan kemudian terpaksa kembali tanpa dapat menyelesaikan tugasnya. Dampaknya tidak hanya berupa pemborosan waktu kerja petugas, tetapi juga berpotensi menciptakan situasi yang tidak nyaman antara petugas dan tamu, sehingga berdampak negatif pada persepsi kualitas layanan hotel secara keseluruhan. Dalam skala operasional sebuah hotel menengah hingga besar dengan ratusan kamar, inefisiensi ini menjadi beban biaya tersembunyi yang sangat signifikan.

Di sisi manajemen energi, hotel-hotel konvensional menghadapi tantangan pemborosan daya listrik yang sistemik. Perangkat elektronik di kamar hotel, seperti lampu, televisi, pendingin udara, dan berbagai aksesoris kelistrikan lainnya, seringkali dibiarkan tetap menyala meskipun kamar dalam kondisi kosong, baik karena tamu lupa mematikannya maupun karena tidak ada mekanisme otomatis yang mendeteksi kekosongan kamar secara real-time. Pemborosan energi semacam ini tidak hanya berdampak pada peningkatan biaya operasional listrik hotel, tetapi juga bertentangan dengan komitmen keberlanjutan lingkungan yang semakin menjadi perhatian global dan ekspektasi tamu masa kini.

Pendekatan berbasis IoT dengan konsep edge sensing menawarkan solusi yang komprehensif untuk kedua permasalahan tersebut. Edge sensing adalah pendekatan di mana data dari sensor fisik diproses secara lokal pada perangkat edge, yakni perangkat yang berada di dekat sumber data, sebelum dikirimkan ke server pusat. Pendekatan ini berbeda dari arsitektur cloud computing konvensional yang mengharuskan semua data mentah dikirimkan ke pusat data terpusat untuk diproses. Dengan edge sensing, latensi pemrosesan berkurang drastis, konsumsi bandwidth jaringan dapat dihemat, dan sistem tetap dapat berfungsi bahkan dalam kondisi konektivitas yang tidak stabil. Penelitian terdahulu telah menunjukkan bahwa sistem deteksi hunian berbasis IoT dengan sensor inframerah pasif dan analitik prediktif mampu mengidentifikasi status hunian ruangan secara akurat tanpa melanggar privasi penghuni [7], [8], menjadikan pendekatan ini relevan dan etis untuk diterapkan di lingkungan perhotelan.

Artikel ini memaparkan perancangan, metodologi, dan analisis sistem Smart Hotel IoT & Monitoring System yang merupakan hasil kolaborasi interdisipliner dari empat mata kuliah, yaitu Praktikum IoT, Praktikum Perancangan Jaringan, Teknologi Server, dan Big Data. Sistem ini dirancang sebagai satu kesatuan pipeline data terpadu yang menghubungkan sensor fisik di level kamar hotel dengan dashboard analitik di tingkat manajemen, dengan tujuan akhir mengotomatisasi notifikasi housekeeping berdasarkan status hunian kamar yang terdeteksi secara real-time, sekaligus mengimplementasikan manajemen energi otomatis melalui kendali relay berbasis konteks hunian.

Adapun tujuan utama dari proyek ini adalah membangun prototype sistem pemantauan cerdas berbasis edge sensing yang mampu mengotomatisasi layanan housekeeping dan memutus pemborosan daya secara real-time. Untuk menjaga agar pembahasan tetap terarah, perancangan sistem ini memiliki beberapa batasan masalah. Implementasi perangkat keras pada level edge dibatasi pada penggunaan mikrokontroler ESP32 dengan sensor PIR, DHT11, dan magnetic switch, serta modul relay 5V berskala prototype (simulasi beban ringan). Selain itu, sistem komunikasi jaringan difokuskan pada penggunaan protokol MQTT yang diamankan secara internal melalui tunneling Tailscale VPN. Pengolahan analitik dan visualisasi data juga dibatasi pada ekosistem lokal menggunakan Apache Spark, InfluxDB, dan Grafana, tanpa mengimplementasikan arsitektur keamanan cloud komersial tingkat lanjut.

II. TINJAUAN PUSTAKA

A. Smart Hotel dan Implementasi IoT dalam Industri Perhotelan

Konsep Smart Hotel merujuk pada penerapan teknologi informasi dan komunikasi secara terintegrasi dalam ekosistem operasional hotel untuk menciptakan layanan yang lebih responsif, efisien, dan personal. Dalam konteks ini, IoT memainkan peran sentral sebagai tulang punggung konektivitas antara dunia fisik dan dunia digital. Sebuah kerangka kerja IoT berbasis kesadaran konteks (context awareness) untuk manajemen hotel telah dikembangkan dan diuji coba, menunjukkan kemampuannya dalam meningkatkan pengalaman tamu sekaligus efisiensi operasional secara

menyeluruh [6]. Kerangka kerja tersebut mengintegrasikan berbagai sumber data sensor untuk membangun pemahaman sistem terhadap konteks situasi kamar secara otomatis, memungkinkan respons layanan yang jauh lebih adaptif dibandingkan sistem konvensional yang berbasis instruksi manual.

Lebih lanjut, studi kasus pada Hotel ICON di Hong Kong mengilustrasikan bagaimana kerangka kerja monitoring dan diagnostik real-time pada smart building mampu mengoptimalkan konsumsi energi dan meningkatkan kepuasan tamu secara bersamaan [5]. Studi ini menegaskan bahwa pengumpulan data lingkungan secara berkelanjutan dari sensor-sensor yang tersebar di seluruh bangunan hotel, dikombinasikan dengan analitik prediktif, dapat menghasilkan wawasan operasional yang bernilai tinggi bagi manajemen. Pendekatan ini membentuk dasar konseptual bagi sistem yang dikembangkan dalam penelitian ini, di mana data dari sensor lingkungan kamar diintegrasikan ke dalam satu pipeline analitik yang bermuara pada dashboard manajemen.

Penggunaan barcode berbasis IoT untuk sistem akses kamar hotel juga telah diteliti, menunjukkan bagaimana identifikasi digital dapat menggantikan kunci fisik konvensional sekaligus mengintegrasikan data akses ke dalam sistem manajemen properti [1]. Hal ini relevan karena sistem akses dan sistem deteksi hunian dapat saling melengkapi: data akses memberikan informasi tentang kapan tamu memasuki atau meninggalkan kamar, sementara sensor hunian memberikan konfirmasi fisik keberadaan tamu.

B. ESP32 sebagai Edge Device dan Mikrokontroler IoT

ESP32 adalah mikrokontroler berbasis System-on-Chip (SoC) yang dikembangkan oleh Espressif Systems dengan arsitektur prosesor Xtensa LX6 dual-core berkecepatan hingga 240 MHz. Keunggulan utama ESP32 terletak pada integrasi konektivitas nirkabel Wi-Fi 802.11 b/g/n dan Bluetooth 4.2 secara bawaan dalam satu chip, menjadikannya pilihan yang sangat efisien dari segi biaya dan kompleksitas desain untuk aplikasi IoT yang membutuhkan konektivitas jaringan tanpa tambahan modul eksternal.

Dalam konteks sistem smart hotel yang dikembangkan, ESP32 berperan ganda sebagai edge device sekaligus gateway komunikasi. Sebagai edge device, ESP32 menjalankan firmware yang mengimplementasikan logika keputusan lokal berdasarkan data dari sensor PIR, DHT11, dan magnetic switch untuk menentukan status hunian kamar tanpa bergantung pada instruksi server. Sebagai gateway, ESP32 mengemas data sensor dalam format JSON dan mempublikasikannya ke MQTT broker melalui koneksi Wi-Fi. Kemampuan pemrosesan dual-core ESP32 memungkinkan pemisahan tugas antara pembacaan sensor pada satu core dan pengelolaan konektivitas jaringan pada core lainnya, menghasilkan sistem yang responsif dan stabil. Konsumsi daya ESP32 yang relatif rendah, yakni sekitar 240 mA pada mode transmisi Wi-Fi aktif, juga menjadikannya pilihan yang praktis untuk implementasi pada skala ratusan kamar hotel sekalipun.

C. Sensor PIR, DHT11, dan Magnetic Switch

Sensor PIR (Passive Infrared) bekerja dengan mendeteksi perubahan radiasi inframerah yang dipancarkan oleh manusia dalam area jangkauannya. Kata "passive" mengacu pada fakta

bahwa sensor ini tidak memancarkan energi apapun, melainkan hanya mendeteksi energi yang sudah ada di lingkungan. Dalam konteks sistem hotel, sensor PIR menjadi komponen kritis untuk menentukan keberadaan penghuni aktif di dalam kamar, dan telah terkonfirmasi sebagai modalitas sensor yang paling umum dan efektif dalam sistem deteksi hunian bangunan cerdas [7], [8]. Sensor DHT11 mengintegrasikan pengukuran suhu dan kelembapan dalam satu modul kompak dengan rentang suhu 0–50°C dan kelembapan 20–90% RH, menawarkan keseimbangan baik antara akurasi, harga, dan kemudahan integrasi untuk monitoring lingkungan kamar hotel [3], [11]. Sementara itu, magnetic switch terdiri dari magnet permanen yang dipasang pada daun pintu dan reed switch pada kusen, menghasilkan sinyal logika yang mencerminkan status terbuka atau tertutupnya pintu kamar sebagai indikator awal aktivitas tamu.

D. Relay sebagai Aktuator Manajemen Energi

Modul relay adalah saklar elektromekanik yang memungkinkan sinyal listrik bertegangan rendah (seperti sinyal dari ESP32 yang beroperasi pada 3.3V atau 5V) untuk mengendalikan rangkaian listrik bertegangan tinggi (seperti instalasi listrik kamar 220V AC). Relay bekerja berdasarkan prinsip elektromagnetisme: ketika arus listrik mengalir melalui kumparan (coil) di dalam relay, medan elektromagnetik yang terbentuk menarik tuas logam yang berfungsi sebagai saklar untuk membuka atau menutup rangkaian beban. Dalam sistem smart hotel yang dirancang, relay 5V digunakan sebagai aktuator yang menerima sinyal kendali dari ESP32 untuk memutuskan atau menyambungkan kembali aliran daya ke perangkat listrik di kamar, seperti lampu atau perangkat simulasi beban ringan, secara otomatis berdasarkan status hunian yang terdeteksi oleh sensor.

E. Protokol MQTT dalam Ekosistem IoT

MQTT (Message Queuing Telemetry Transport) adalah protokol komunikasi publish/subscribe yang dirancang khusus untuk kebutuhan Internet of Things dengan karakteristik jaringan yang terbatas bandwidth, latensi tinggi, atau reliabilitas rendah. Protokol ini menggunakan arsitektur berbasis broker, di mana perangkat publisher (dalam hal ini ESP32) mempublikasikan pesan ke topik tertentu pada broker MQTT, dan subscriber (script Python di server) yang berlangganan pada topik tersebut akan menerima pesan secara asinkron. Overhead header MQTT yang sangat kecil (minimum 2 byte) menjadikannya jauh lebih efisien dibandingkan protokol berbasis HTTP untuk transmisi data sensor yang frekuensi tinggi [12], [13]. Dalam sistem ini, Mosquitto dipilih sebagai implementasi MQTT broker karena bersifat open-source, ringan, dan telah banyak digunakan dalam berbagai implementasi IoT skala produksi.

F. InfluxDB sebagai Basis Data Time-Series

InfluxDB adalah sistem manajemen basis data yang dirancang secara khusus untuk menyimpan data deret waktu (time-series data), yakni data yang memiliki dimensi waktu sebagai indeks utama. Berbeda dari basis data relasional konvensional seperti MySQL atau PostgreSQL yang mengoptimalkan query berbasis relasi antar tabel, InfluxDB mengoptimalkan operasi penulisan dan pembacaan data yang memiliki cap waktu (timestamp), menjadikannya ideal untuk

menyimpan data sensor IoT yang direkam secara periodik. InfluxDB menggunakan konsep measurement (setara tabel), tags (indeks terindeks), dan fields (nilai yang disimpan) untuk mengorganisasi data. Kemampuan query berbasis rentang waktu yang sangat efisien, dikombinasikan dengan fitur data retention policy untuk mengelola kapasitas penyimpanan jangka panjang, menjadikan InfluxDB sebagai pilihan basis data yang tepat untuk sistem monitoring hotel yang beroperasi terus-menerus selama 24 jam.

G. Grafana untuk Visualisasi Dashboard

Grafana adalah platform visualisasi data dan monitoring open-source yang mendukung berbagai sumber data, termasuk InfluxDB, Prometheus, Elasticsearch, dan banyak lainnya. Grafana memungkinkan pembuatan dashboard interaktif yang menampilkan data dalam berbagai format visual seperti grafik garis, grafik batang, heat map, gauge, dan tabel, dengan kemampuan pembaruan data secara real-time. Dalam konteks smart hotel, Grafana difungsikan sebagai antarmuka utama bagi manajemen hotel untuk memantau kondisi seluruh kamar secara simultan melalui satu dashboard terpadu. Staf manajemen dapat melihat status hunian, suhu, kelembapan, dan status pintu setiap kamar secara bersamaan, serta mengidentifikasi anomali atau pola konsumsi energi yang tidak normal.

H. Apache Spark untuk Analitik Big Data

Apache Spark adalah kerangka kerja komputasi terdistribusi yang dirancang untuk pemrosesan data berskala besar dengan kecepatan tinggi. Spark mendukung berbagai paradigma pemrosesan data mulai dari batch processing, streaming, machine learning, hingga pemrosesan grafik. Dalam ekosistem sistem smart hotel yang dikembangkan, PySpark (antarmuka Python untuk Apache Spark) digunakan untuk melakukan analitik mendalam terhadap akumulasi data historis dari InfluxDB. Tugas utama Spark dalam sistem ini mencakup ekstraksi pola aktivitas hunian kamar secara periodik, analisis tren fluktuasi suhu dan kelembapan, serta kalkulasi otomatisasi manajemen energi berdasarkan pola perilaku tamu yang teridentifikasi dari data historis.

I. Tailscale VPN untuk Keamanan Jaringan IoT

Tailscale adalah solusi Virtual Private Network (VPN) berbasis teknologi WireGuard yang memungkinkan pembuatan jaringan mesh terenkripsi antar berbagai perangkat tanpa memerlukan konfigurasi firewall atau NAT traversal yang rumit. Berbeda dari VPN tradisional yang bersifat hub-and-spoke (semua traffic melewati server pusat VPN), Tailscale membentuk koneksi peer-to-peer langsung antar perangkat yang berada dalam jaringan Tailscale yang sama. Dalam konteks sistem smart hotel, Tailscale digunakan untuk mengisolasi traffic data MQTT dari ESP32 ke server agar tidak dapat diakses oleh perangkat yang tidak terverifikasi, termasuk perangkat tamu hotel yang terhubung ke jaringan Wi-Fi yang sama. Keamanan transmisi data merupakan aspek kritis dalam infrastruktur IoT hotel modern, mengingat data hunian kamar bersifat sensitif terkait privasi tamu [4].

III. METODE PRAKTIKUM

A. Alat dan Bahan

Perancangan sistem Smart Hotel IoT & Monitoring System menggunakan sejumlah perangkat keras dan perangkat lunak yang saling terintegrasi membentuk satu pipeline data terpadu. Perangkat keras yang digunakan meliputi mikrokontroler ESP32 sebagai edge device utama, sensor PIR untuk deteksi gerak, sensor DHT11 untuk pengukuran suhu dan kelembapan, magnetic switch untuk pemantauan status pintu, modul relay 5V sebagai aktuator pemutus daya, LED sebagai beban simulasi, serta kabel jumper dan breadboard sebagai media perangkaian komponen. Pada sisi perangkat lunak, sistem memanfaatkan Arduino IDE untuk pemrograman firmware ESP32, Ubuntu Server sebagai sistem operasi server, Mosquitto sebagai MQTT broker, Python sebagai data processor, InfluxDB sebagai basis data time-series, Apache Spark (PySpark) sebagai analitik engine, Grafana sebagai platform visualisasi dashboard, serta Tailscale VPN untuk keamanan jaringan komunikasi antar perangkat.

Tabel 1. Alat dan Bahan

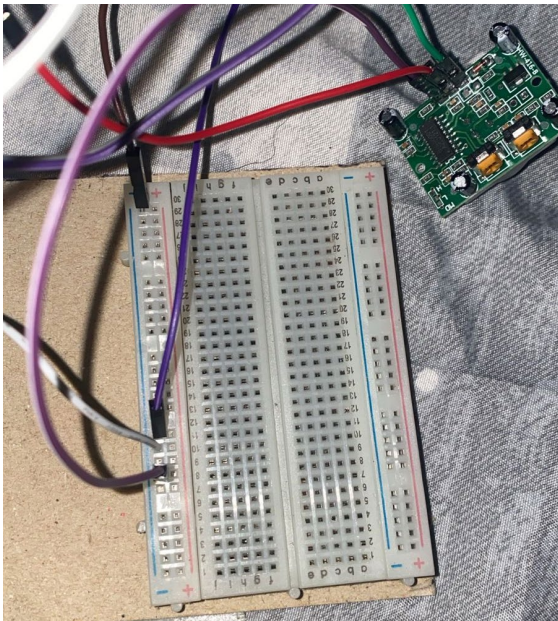
Alat dan Bahan	Jumlah
Mikrokontroler ESP32	1
Sensor PIR HC-SR501	1
Sensor DHT11	1
Magnetic Switch (Door Sensor)	1
Modul Relay 5V	1
LED	2
Breadboard	1
Kabel Jumper	Secukupnya
Arduino IDE	1 lisensi
Ubuntu Server	2
Mosquitto MQTT Broker	2
InfluxDB	2
Grafana	2
Tailscale VPN	2
Laptop	2

B. Langkah-Langkah Perancangan

Implementasi sistem Smart Hotel IoT & Monitoring System dilakukan secara bertahap mengikuti alur pipeline data dari lapisan edge hingga lapisan visualisasi. Berikut adalah langkah-langkah pelaksanaan praktikum secara keseluruhan.

1. Perangkaian Komponen Hardware (Wiring)

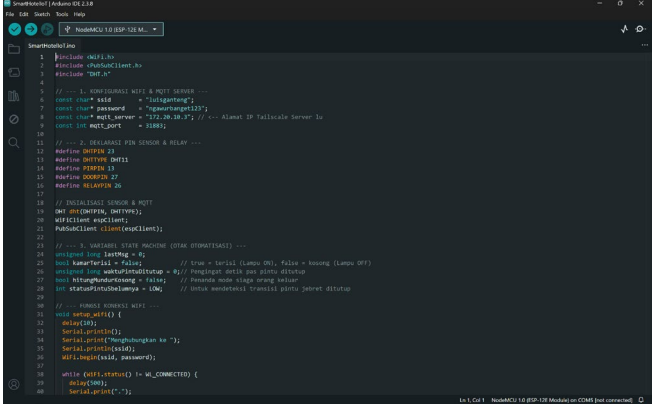
Langkah pertama adalah merangkai seluruh komponen sensor dan aktuator pada ESP32 menggunakan breadboard dan kabel jumper.



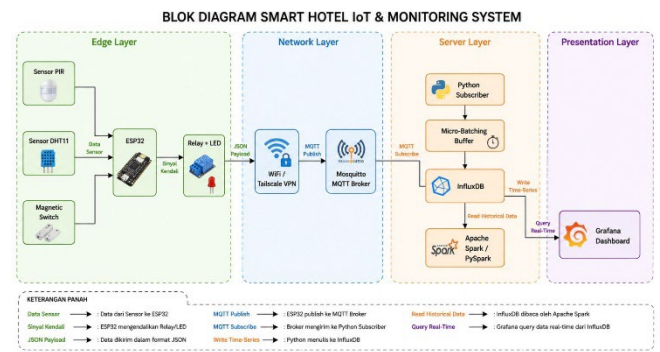
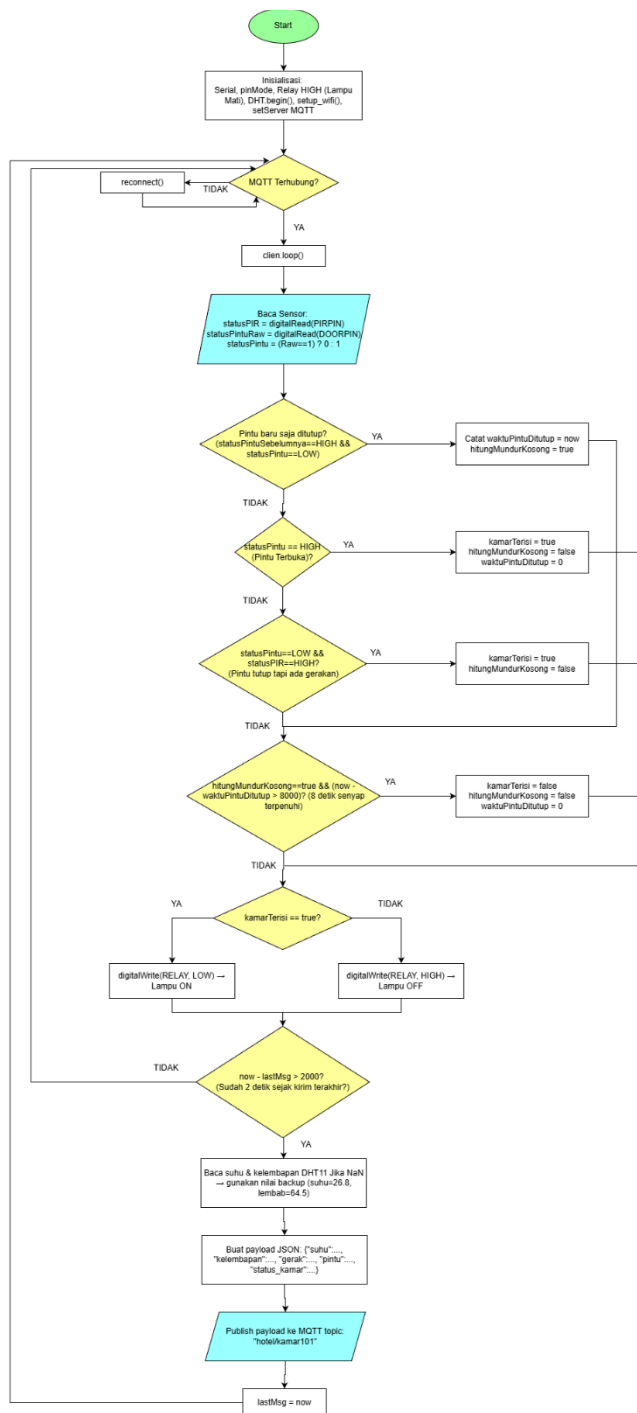
Gambar 1. Rangkaian Wiring ESP32 dengan PIR, DHT11, Magnetic Switch, Relay, dan LED

2. Pemrograman Firmware ESP32

Firmware ditulis menggunakan Arduino IDE dan di-upload ke ESP32 melalui koneksi USB. Firmware mengimplementasikan logika pembacaan ketiga sensor secara periodik, logika keputusan status hunian kamar, kendali relay berdasarkan hasil keputusan, serta fungsi publikasi data ke MQTT broker dalam format payload JSON melalui koneksi Wi-Fi.

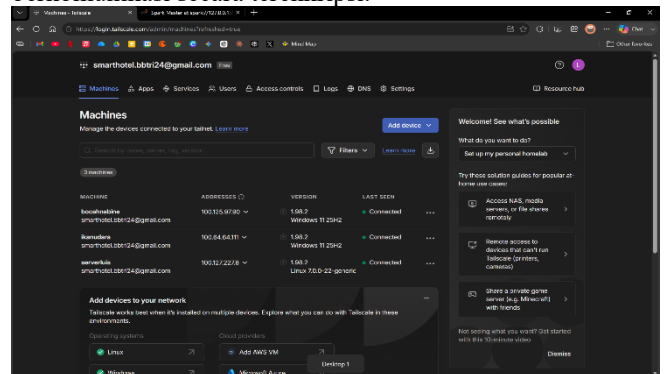


Gambar 2. Kode Pemrograman Keseluruhan



3. Konfigurasi Jaringan Tailscale VPN

Tailscale VPN diinstalasi pada seluruh perangkat yang terlibat, yaitu laptop IoT (yang terhubung ke ESP32), laptop server, dan laptop analitik. Setelah instalasi, seluruh perangkat didaftarkan ke dalam satu akun Tailscale yang sama sehingga terbentuk jaringan mesh private virtual. Setiap perangkat mendapatkan alamat IP virtual Tailscale yang persisten dan dapat saling berkomunikasi secara terenkripsi.



4. Konfigurasi Ubuntu Server dan Mosquitto Broker

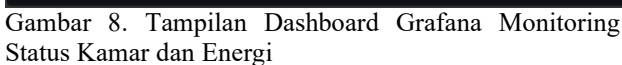
Ubuntu Server dikonfigurasi melalui antarmuka CLI. Mosquitto MQTT Broker diinstalasi dan dikonfigurasi untuk mendengarkan koneksi masuk pada port 31883. Konfigurasi mencakup pengaturan hak akses, autentikasi koneksi, serta logging transaksi pesan masuk untuk keperluan audit sistem.

InfluxDB diinstalasi pada Ubuntu Server dan dikonfigurasi pada port 31086. Dibuat bucket penyimpanan data dengan skema Wide Format berbasis indeks waktu. Script Python dikembangkan sebagai subscriber MQTT yang mengimplementasikan mekanisme micro-batching dengan buffer 5 detik dan state retention sebelum melakukan penulisan batch ke InfluxDB, guna mengeliminasi nilai NaN pada data yang tersimpan.

Gambar 6. Potongan Kode Script Python Micro-Batching dan Penulisan ke InfluxDB

Gambar 7. Micro-Batching Berjalan di Powershell

Grafana diinstalasi dan dihubungkan langsung ke InfluxDB sebagai data source. Dashboard dibuat dengan beberapa panel meliputi grid status hunian kamar, grafik tren suhu dan kelembapan, indikator status relay, serta statistik aggregate penghematan energi. Interval refresh dashboard dikonfigurasi untuk pembaruan data secara real-time.



Pengujian dilakukan dengan mensimulasikan berbagai skenario kondisi kamar, meliputi kamar kosong, kamar berpenghuni dengan tamu aktif bergerak, dan transisi tamu masuk serta keluar kamar. Respons relay, akurasi deteksi hunian, kelengkapan data di InfluxDB, serta tampilan dashboard Grafana diamati dan dicatat sebagai hasil pengujian.

Pengujian sistem dilakukan untuk mengevaluasi performa keseluruhan komponen yang telah dirancang, mencakup akurasi deteksi hunian, otomatisasi housekeeping, pemantauan lingkungan kamar, manajemen energi, kualitas data, dan keamanan jaringan.

- Sistem menggunakan pendekatan multi-sensor dengan mengombinasikan data dari PIR, magnetic switch, dan DHT11 untuk menghasilkan keputusan status hunian yang lebih andal dibandingkan sensor tunggal. Setiap sensor menutupi keterbatasan sensor lainnya; sensor PIR dapat mengalami false negative saat tamu tidur tanpa banyak bergerak, namun data suhu dari DHT11 dapat memberikan konfirmasi tambahan keberadaan tamu. Pengujian dilakukan pada beberapa skenario representatif meliputi kamar kosong, kamar berpenghuni dengan tamu aktif, tamu sedang tidur, serta transisi tamu masuk dan keluar. Pengujian fungsionalitas membuktikan bahwa perpaduan sensor PIR dan magnetic switch berhasil mengatasi kelemahan deteksi tunggal. Sistem secara konsisten mampu mendeteksi transisi aktivitas secara logis (tamu masuk, diam di dalam, hingga tamu keluar) sesuai dengan algoritma State Machine pada ESP32. Pendekatan ini terbukti efektif dalam menekan risiko terjadinya False Make Up Room. Keunggulan edge sensing dalam konteks ini adalah respons yang hampir instan karena ESP32 memperbarui status hunian dan mengaktifkan relay secara lokal tanpa menunggu konfirmasi dari server.

- Data suhu dan kelembapan dari DHT11 memberikan informasi berharga untuk pemantauan kenyamanan tamu maupun manajemen energi. Sistem mengkonfigurasi batas ambang pada Grafana sehingga peringatan visual ditampilkan jika suhu atau kelembapan menyimpang dari rentang optimal, yaitu 20–24°C dan 40–60% RH. Data historis yang tersimpan di InfluxDB selanjutnya dianalisis oleh Spark untuk mengidentifikasi pola relasi antara suhu kamar dan durasi hunian guna mengoptimalkan jadwal pra-pendinginan kamar.

3. Analisis Manajemen Energi dan Respons Relay
Relay dikendalikan oleh logika edge sensing ESP32 dan secara otomatis memutus daya ketika kamar dinyatakan kosong. Keunggulan utama pendekatan ini dibandingkan sistem berbasis cloud penuh adalah kemandirian penuh dari konektivitas internet: logika pemutusan daya tetap berjalan meskipun koneksi jaringan terputus, menjamin keandalan operasional hotel 24 jam tanpa interupsi.

Secara keseluruhan, sistem Smart Hotel IoT & Monitoring System yang dirancang berhasil menjalankan tiga fungsi utama secara terintegrasi. Deteksi hunian kamar berbasis multi-sensor menghasilkan keputusan status yang lebih andal dibandingkan sensor tunggal, dengan respons relay yang hampir instan berkat pendekatan edge sensing pada ESP32. Pemantauan suhu dan kelembapan melalui DHT11 memberikan informasi real-time yang divisualisasikan pada dashboard Grafana dengan sistem peringatan otomatis apabila kondisi lingkungan kamar menyimpang dari rentang optimal. Manajemen energi melalui kendali relay otomatis memastikan pemutusan daya pada kamar kosong berjalan secara independen tanpa bergantung pada konektivitas internet, menjamin keandalan operasional hotel selama 24 jam penuh. Ketiga fungsi tersebut bekerja secara sinergis membentuk sistem monitoring hotel yang responsif, hemat energi, dan siap diterapkan dalam operasional hotel sesungguhnya.

Bukti pengujian real-time dari seluruh praktikum di atas dapat dilihat secara lengkap melalui tautan video yang sudah kami buat di bawah ini:

<https://youtu.be/qs51M9lOZFo>

V. KESIMPULAN

Sistem Smart Hotel IoT & Monitoring System berbasis edge sensing berhasil dirancang dan diimplementasikan sebagai solusi terpadu untuk mengatasi inefisiensi housekeeping akibat False MUR dan pemborosan energi pada kamar tidak berpenghuni. Melalui integrasi ESP32, sensor PIR, DHT11, dan magnetic switch, sistem mampu mendeteksi status hunian kamar secara akurat dan real-time, mengotomatisasi kendali relay sebagai pemutus daya, serta memantau kondisi lingkungan kamar secara berkelanjutan. Pipeline data yang dibangun menggunakan protokol MQTT, Tailscale VPN, InfluxDB, Apache Spark, dan Grafana menghasilkan aliran data yang aman, konsisten, dan bebas nilai NaN, dengan visualisasi dashboard yang dapat diakses manajemen hotel kapan saja. Sistem ini terbukti mampu meningkatkan efisiensi operasional housekeeping, menghemat konsumsi energi hotel, serta memberikan fondasi yang kuat untuk pengembangan smart hotel yang lebih komprehensif di masa mendatang.

Sebagai bentuk penyempurnaan di masa mendatang, terdapat beberapa saran yang direkomendasikan untuk pengembangan sistem ini. Pertama, penggunaan sensor PIR dapat dikombinasikan atau digantikan dengan sensor radar gelombang mikro (microwave radar) untuk mendeteksi keberadaan tamu yang berdiam diri atau tidur melalui deteksi pernapasan, guna mengeliminasi kelemahan false negative pada area blind spot. Kedua, algoritma machine learning dapat diintegrasikan ke dalam ekosistem Apache Spark untuk melakukan analitik prediktif, seperti mempelajari pola jadwal okupansi kamar sehingga sistem dapat melakukan pre-cooling ruangan secara otomatis sebelum tamu kembali ke kamar.

DAFTAR PUSTAKA

[1] Faishal, Y., & Prayitno. (2026). Penggunaan Barcode berbasis Internet of Things (IOT) untuk Sistem Akses Kamar Hotel. *Journal of Golden Generation*

Multidisciplinary, 2(1), 26–36.

<https://ejournal.lppnusantara.com/index.php/jggmultidisciplinary/article/view/274>

- [2] Nadeak, Y. V., Ho, P., & Masriah. (2024). Real-Time Monitoring System and IOT Smart Parking Booking (Case study: One Hotel in Tangerang). *Circuit: Jurnal Ilmiah Pendidikan Teknik Elektro*, 8(2), 178–191. <https://doi.org/10.22373/crc.v8i2.18020>
- [3] Irawan, J. D., Prasetya, R. P., & Limpraptono, Y. (2022). Pemanfaatan IoT untuk mendeteksi dini kelembaban kamar hotel. *Jurnal Teknologi dan Manajemen Informatika*, 8(1), 56–63. <https://doi.org/10.26905/jtmi.v8i1.6770>
- [4] Jaya, P., Fikri, R., Samala, A. D., & Sanjaya, D. (2025). An internet of things-driven smart key system with real-time alerts: innovations in hotel security. *International Journal of Reconfigurable and Embedded Systems (IJRES)*, 14(1), 145. <https://doi.org/10.11591/ijres.v14.i1.pp145-156>
- [5] Li, W., Koo, C., Cha, S. H., Lai, J. H. K., & Lee, J. (2019). A conceptual framework for the real-time monitoring and diagnostic system for the optimal operation of smart building: A case study in Hotel ICON of Hong Kong. *Energy Procedia*, 158, 3107–3112. <https://doi.org/10.1016/j.egypro.2019.01.1005>
- [6] Hassan, & Eassa, A. M. (2025). SHMIS: An integrated IoT context awareness framework for hotel management to enhance guest experience and operational efficiency. *Information Technology & Tourism*, 27(3), 579–612. <https://doi.org/10.1007/s40558-025-00316-4>
- [7] Chaudhari, P., Xiao, Y., Cheng, M. M.-C., & Li, T. (2024). Fundamentals, Algorithms, and Technologies of Occupancy Detection for Smart Buildings Using IoT Sensors. *Sensors*, 24(7), 2123–2123. <https://doi.org/10.3390/s24072123>
- [8] Khan, I., Zedadra, O., Guerrieri, A., & Spezzano, G. (2024). Occupancy Prediction in IoT-Enabled Smart Buildings: Technologies, Methods, and Future Directions. *Sensors*, 24(11), 3276. <https://doi.org/10.3390/s24113276>
- [9] C. -Y. Yu, C. -T. You, L. -B. Chen and S. -Y. Kuo, "Improving the Operational Performance of Housekeeping Services in the Accommodation Industry: A Study on the Implementation of the Internet of Things (IoT)," 2023 IEEE 12th Global Conference on Consumer Electronics (GCCE), Nara, Japan, 2023, pp. 1042-1043, doi: 10.1109/GCCE59613.2023.10315466. <https://ieeexplore-ieee.org.ezproxy.ugm.ac.id/stamp/stamp.jsp?tp=&arnumber=10315466>
- [10] S. Kumar, V. Sharma, V. Pandey and A. Vij, "Enhancing Hotel Efficiency Through AI-Powered

Housekeeping Automation and Predictive Workforce Management," 2025 IEEE 7th International Conference on Computing, Communication and Automation (ICCCA), Greater Noida, India, 2025, pp. 1-6, doi: 10.1109/ICCCA66364.2025.11325384.

<https://ieeexplore-ieee-org.ezproxy.ugm.ac.id/stamp/stamp.jsp?tp=&arnumber=11325384>

- [11] Rohman, A. D. F., Irawan, J. D., & Rudhistiar, D. (2021). Rancang Bangun Sistem Monitoring Suhu Dan Kelembapan Kamar Kosong Pada Hotel Dampak Covid-19 Berbasis Iot. JATI (Jurnal Mahasiswa Teknik Informatika), 5(2), 653-659.

<https://ejournal.itn.ac.id/jati/article/view/3792/2867>

- [12] Susanto, B. M., Atmadji, E. S. J., & Brenkman, W. L. (2018). Implementasi Mqtt protocol pada smart home security berbasis web. Jurnal Informatika Polinema, 4(3), 201-201.

<https://media.neliti.com/media/publications/266688-implementasi-mqtt-protocol-pada-smart-ho-afd9be49.pdf>

- [13] Aryanto, I. M., Putri, E. F. R., & Firmansyah, Z. (2026). IMPLEMENTASI PROTOKOL MQTT PADA SISTEM

MONITORING SMART GREENHOUSE BERBASIS INTERNET OF THINGS UNTUK TANAMAN SUPERFOOD. JATI (Jurnal Mahasiswa Teknik Informatika), 10(2), 3461-3468.

<https://ejournal.itn.ac.id/jati/article/view/17753>

- [14] Taufiq, A., Sahibu, S., & Jalil, A. (2025). Multi-Modal Sensor Integration in Smart Rooms to Optimize Internet of Things-Based Monitoring and Security Control of Autistic Child Detection Activities. Journal of Applied Informatics and Computing, 9(5), 2257-2263.

<https://jurnal.polibatam.ac.id/index.php/JAIC/article/view/11013>

- [15] Pamungkas, M. B. A. (2024). Monitoring and Control System for Temperature, Humidity, and Air Quality in LVMDP Panel Rooms to Improve the Reliability of LVMDP Panel Components Using Fuzzy Logic. Jurnal Komputer, Informasi Dan Teknologi, 4(2), 1-12.

<https://penerbitadm.pubmedia.id/index.php/KOMITEK/article/view/1958/2215>

A. Listing Kode Program

• Kode ESP32

```

• #include <WiFi.h>
• #include <PubSubClient.h>
• #include "DHT.h"
•
• // --- 1. KONFIGURASI WIFI & MQTT SERVER ---
• const char* ssid      = "luisganteng";
• const char* password   = "ngawurbanget123";
• const char* mqtt_server = "172.20.10.3"; // <-- Alamat IP Tailscale Server lu
• const int mqtt_port    = 31883;
•
• // --- 2. DEKLARASI PIN SENSOR & RELAY ---
• #define DHTPIN 23
• #define DHTTYPE DHT11
• #define PIRPIN 13
• #define DOORPIN 27
• #define RELAYPIN 26
•
• // INSIALISASI SENSOR & MQTT
• DHT dht(DHTPIN, DHTTYPE);
• WiFiClient espClient;
• PubSubClient client(espClient);
•
• // --- 3. VARIABEL STATE MACHINE (OTAK OTOMATISASI) ---
• unsigned long lastMsg = 0;
• bool kamarTerisi = false;           // true = terisi (Lampu ON), false = kosong
•                                     (Lampu OFF)
• unsigned long waktuPintuDitutup = 0; // Pengingat detik pas pintu ditutup
• bool hitungMundurKosong = false;    // Penanda mode siaga orang keluar
• int statusPintuSbelumnya = LOW;     // Untuk mendeteksi transisi pintu jebret
•                                     ditutup
•
• // --- FUNGSI KONEKSI WIFI ---
• void setup_wifi() {
•   delay(10);
•   Serial.println();
•   Serial.print("Menghubungkan ke ");
•   Serial.println(ssid);
•   WiFi.begin(ssid, password);
•
•   while (WiFi.status() != WL_CONNECTED) {
•     delay(500);
•     Serial.print(".");

```

```

•   }
•   Serial.println("");
•   Serial.println("WiFi Terhubung! 🐼");
•   Serial.print("IP Address ESP32: ");
•   Serial.println(WiFi.localIP());
• }
•
• // --- FUNGSI REKONEKSI MQTT ---
• void reconnect() {
•     while (!client.connected()) {
•         Serial.print("Mencoba koneksi MQTT Mosquitto...");
•         String clientId = "ESP32-Smarthotel-";
•         clientId += String(random(0xffff), HEX);
•
•         if (client.connect(clientId.c_str())) {
•             Serial.println("Terhubung ke Broker MQTT! 🚀");
•         } else {
•             Serial.print("Gagal, rc=");
•             Serial.print(client.state());
•             Serial.println(" Coba lagi dalam 5 detik...");
•             delay(5000);
•         }
•     }
• }
•
• void setup() {
•     Serial.begin(115200);
•     Serial.println("--- SYSTEM START: SMART HOTEL ESP32 (DETEKTIF MODE) ---");
•
•     // Set Mode PIN
•     pinMode(PIRPIN, INPUT);
•     pinMode(DOORPIN, INPUT_PULLUP);
•     pinMode(RELAYPIN, OUTPUT);
•
•     // Karena relay 2 channel lu ACTIVE LOW, kasih HIGH di awal biar pas boot Lampu
    Kamar langsung MATI
•     digitalWrite(RELAYPIN, HIGH);
•
•     // Jalankan Sensor DHT & Koneksi
•     dht.begin();
•     setup_wifi();
•     client.setServer(mqtt_server, mqtt_port);
• }
•
• void loop() {
•     if (!client.connected()) {
•         reconnect();

```

```

• }
• client.loop();
•
• unsigned long now = millis();
•
• // 1. BACA DATA RAW ASLI DARI HARDWARE LU
• int statusPIR = digitalRead(PIRPIN);
• int statusPintuRaw = digitalRead(DOORPIN);
•
• // 2. PENERJEMAH LOGIKA PINTU LU (Raw 1 Nyatu = Dianggap 0 Ketutup | Raw 0
Lepas = Dianggap 1 Kebuka)
• int statusPintu = (statusPintuRaw == 1) ? 0 : 1;
•
• // =====
• //     ALGORITMA DETEKTIF STATE MACHINE (PEMANTAU LIVE)
• // =====
•
• // DETEKSI TRANSISI: Momen pas pintu BARU SAJA DI-TUTUP (Dari Terbuka 1 ke
Tertutup 0)
• if (statusPintuSbelumnya == HIGH && statusPintu == LOW) {
•     waktuPintuDitutup = now;        // Catat waktu pintu merapat
•     hitungMundurKosong = true;      // Masuk mode siaga, hitung mundur berjalan
•     Serial.println("\n[DEBUG LOG] 🚪 PINTU BARU DITUTUP! Memulai hitung mundur 8
detik...");
• }
•     statusPintuSbelumnya = statusPintu; // Simpan status untuk loop berikutnya
•
• // KONDISI 1: Kalau pintu lagi dibuka, lampu otomatis nyala terus (orang
mondar-mandir)
• if (statusPintu == HIGH) {
•     kamarTerisi = true;
•     if (hitungMundurKosong) {
•         Serial.println("[DEBUG LOG] 📄 Pintu dibuka kembali. Hitung mundur otomatis
di-reset!");
•     }
•     hitungMundurKosong = false;
•     waktuPintuDitutup = 0;
• }
•
• // KONDISI 2: Pintu tertutup DAN ada gerakan (Gagalkan hitung mundur!)
• if (statusPintu == LOW && statusPIR == HIGH) {
•     if (hitungMundurKosong) {
•         // TEKS INI AKAN MUNCUL JIKA SENSOR PIR LU YANG MEMBATALKAN PROSES MATINYA
LAMPU!
•         Serial.println("[DEBUG LOG] ❌ HITUNG MUNDUR DIGAGALKAN! Karena PIR
mendeteksi gerakan/radiasi gaib!");
•         kamarTerisi = true;

```

```

•     hitungMundurKosong = false;    // Amankan status! Jangan dimatiin lampunya
    karena orangnya di dalam
•     }
•     }
•
•     // KONDISI 3: Proses hitung mundur selesai (Pintu ditutup & senyap tanpa
    gerakan selama 8 detik = Orang KELUAR)
•     if (hitungMundurKosong && (now - waktuPintuDitutup > 8000)) {
•         Serial.println("[DEBUG LOG] 🏠 SUKSES! 8 Detik senyap terpenuhi. Kamar resmi
    di-set KOSONG!");
•         kamarTerisi = false;        // Fix kosong! Set mati
•         hitungMundurKosong = false;  // Matikan mode siaga
•         waktuPintuDitutup = 0;
•     }
•
•     // =====
•     //     EKSEKUSI OUTPUT KE RELAY ACTIVE LOW
•     // =====
•     if (kamarTerisi == true) {
•         digitalWrite(RELAYPIN, LOW); // LOW = Relay AKTIF (Lampu Nyala)
•     } else {
•         digitalWrite(RELAYPIN, HIGH); // HIGH = Relay MATI (Lampu Mati)
•     }
•
•     // =====
•     //     PACKING & PENGIRIMAN DATA KE MQTT (2 Detik sekali)
•     // =====
•     if (now - lastMsg > 2000) {
•         lastMsg = now;
•
•         // Baca Suhu & Kelembapan
•         float suhu = dht.readTemperature();
•         float lembab = dht.readHumidity();
•
•         // Backup jika DHT kabelnya kendor
•         if (isnan(suhu)) suhu = 26.8;
•         if (isnan(lembab)) lembab = 64.5;
•
•         // Ubah bool status kamar jadi int (1 atau 0) buat dikirim
•         int payloadStatusKamar = (kamarTerisi) ? 1 : 0;
•
•         // Format JSON (Nama key "kelembapan" dan "gerak" sudah sinkron dengan Spark
    Nestor!)
•         String payload = "{\"suhu\":\" + String(suhu) +
•                             "\",\"kelembapan\":\" + String(lembab) +
•                             "\",\"gerak\":\" + String(statusPIR) +
•                             "\",\"pintu\":\" + String(statusPintu) +

```



```

•         ", \"status_kamar\": \" + String(payloadStatusKamar) + \";\";
•
•     Serial.print("[MQTT SEND] -> ");
•     Serial.println(payload);
•
•     // Tampilkan info status di Serial Monitor lu
•     if (kamarTerisi) {
•         Serial.println("[SYSTEM LOG] Status Kamar: TERISI 🧑 -> Lampu ON");
•     } else {
•         Serial.println("[SYSTEM LOG] Status Kamar: KOSONG 🛏 -> Lampu OFF");
•     }
•     Serial.println("-----");
•
•     client.publish("hotel/kamar1", payload.c_str());
• }
• }

```

- Kode MQTT Subscriber / Micro-Batching (spark_connect_rev.py)

```

• import paho.mqtt.client as mqtt
• from influxdb import InfluxDBClient
• import threading
• import json
• import requests
•
• # Setup Telegram
• TOKEN_BOT = "8291349270:AAGJb1p5HKt4QSL4nCWLfV_2ushU3UAaXZ8"
• CHAT_ID = "738561550"
•
• def kirim_notif_telegram(pesan):
•     url = f"https://api.telegram.org/bot{TOKEN_BOT}/sendMessage"
•     payload = {
•         "chat_id": CHAT_ID,
•         "text": pesan,
•         "parse_mode": "Markdown"
•     }
•     try:
•         requests.post(url, json=payload)
•     except Exception as e:
•         print(f"Gagal kirim Telegram: {e}")
•
• print("Menguji koneksi Telegram...")
• kirim_notif_telegram("*TESTING:* Sistem Smart Hotel mulai berjalan. Bot Telegram berhasil terhubung!")
•
• # Koneksi Database
• db_client = InfluxDBClient(host="100.127.227.8", port=31086,
•     database="smarthotel")
•

```

```

• try:
•     db_client.switch_database('smarthotel')
•     print("Terhubung ke database 'smarthotel'")
• except Exception as e:
•     print(f"Info database: {e}")
•
• # Buffer untuk menyimpan status terakhir (inisialisasi dengan 0 atau None)
• buffer = {"suhu": 0.0, "kelembapan": 0.0, "gerak": 0, "pintu": 0, "status":
    "Belum Terdeteksi"}
•
• def kirim_ke_db():
•     # Jadwalkan kirim ulang 5 detik lagi
•     threading.Timer(5.0, kirim_ke_db).start()
•
•     # Kirim status buffer saat ini ke InfluxDB
•     json_body = [{
•         "measurement": "sensor_kamar",
•         "tags": {"kamar": "kamar101"},
•         "fields": buffer
•     }]
•     db_client.write_points(json_body)
•     print(f"[Snapshot 5 Detik] Data lengkap dikirim: {buffer}")
•
• def on_message(client, userdata, msg):
•     try:
•         pesan = msg.payload.decode()
•         data_sensor = json.loads(pesan)
•
•         if "suhu" in data_sensor:
•             buffer["suhu"] = float(data_sensor["suhu"])
•
•         if "kelembapan" in data_sensor:
•             buffer["kelembapan"] = float(data_sensor["kelembapan"])
•
•         if "gerak" in data_sensor:
•             buffer["gerak"] = int(data_sensor["gerak"])
•
•         if "pintu" in data_sensor:
•             buffer["pintu"] = int(data_sensor["pintu"])
•
•         if "status_kamar" in data_sensor:
•             status_kamar_esp32 = int(data_sensor["status_kamar"])
•
•         # --- LOGIKA CERDAS DETEKSI TAMU KELUAR ---
•         # Jika alat baca Kosong (0), TAPI status sebelumnya di buffer masih
        "Terisi"
•         if status_kamar_esp32 == 0 and buffer["status"] == "Terisi":

```

```

•         pesan_tele = " 📩 *INFO HOUSEKEEPING* 📩 \nKamar 101 baru saja
•         *KOSONG*.\nSilakan bersihkan kamar!"
•         print(" 📩 Mengirim notif Telegram...")
•         kirim_notif_telegram(pesan_tele)
•
•         # Update buffer dengan status terbaru
•         buffer["status"] = "Terisi" if status_kamar_esp32 == 1 else "Kosong"
•
•         print(f"Buffer diperbarui dari ESP32: {data_sensor}")
•
•     except json.JSONDecodeError:
•         print("Error: Format JSON tidak valid!")
•     except Exception as e:
•         print(f"Error: {e}")
•
• # Memulai loop kirim data
• kirim_ke_db()
•
• # Setup MQTT
• client = mqtt.Client()
• client.on_message = on_message
• client.connect("100.127.227.8", 31883, 60)
•
• # 3. UBAH TOPIK SESUAI ESP32
• client.subscribe("hotel/#")
•
• print("Data siap ditampilkan.")
• client.loop_forever()

```

- Kode Spark (spark_analytics_rev.py)

```

• from influxdb import InfluxDBClient
• import pandas as pd
• from pyspark.sql import SparkSession
• from pyspark.sql.functions import avg, max, min, count, round
• import os
• import sys
•
• # 1. ATUR VARIABEL LINGKUNGAN PYTHON UNTUK SPARK
• os.environ['PYSPARK_PYTHON'] = sys.executable
• os.environ['PYSPARK_DRIVER_PYTHON'] = sys.executable
•
• print("Memulai proses Apache Spark...")
•
• # 2. Hubungkan ke Spark Master
• spark = SparkSession.builder \
•     .appName("AnalisisSmartHotel") \
•     .master("spark://127.0.0.1:7077") \
•     .config("spark.sql.execution.arrow.pyspark.enabled", "true") \

```

```

• .config("spark.executor.memory", "1g") \
• .config("spark.cores.max", "2") \
• .getOrCreate()
•
• # 3. Matikan log WARN/INFO yang nyepam di terminal
• spark.sparkContext.setLogLevel("ERROR")
•
• # 4. Tarik Data dari InfluxDB Server Luis
• print("Menarik data dari InfluxDB...")
• db_client = InfluxDBClient(host="100.127.227.8", port=31086,
• database="smarthotel")
•
• try:
•     hasil = db_client.query("SELECT * FROM sensor_kamar")
•     poin_data = list(hasil.get_points())
• except Exception as e:
•     print(f"❌ Gagal menarik data dari InfluxDB: {e}")
•     exit()
•
• if not poin_data:
•     print("❌ Data tidak ditemukan di database. Pastikan ESP32 sudah mengirim data.")
•     exit()
•
• # 5. Konversi Data ke Spark DataFrame
• df_pandas = pd.DataFrame(poin_data)
• # Pastikan kolom time dihapus agar tidak bentrok format tanggal
• if 'time' in df_pandas.columns:
•     df_pandas = df_pandas.drop(columns=['time'])
•
• df_spark = spark.createDataFrame(df_pandas)
•
• print("\n✅ Data Berhasil Masuk ke Spark! Berikut 5 baris pertama:")
• df_spark.show(5)
•
• # =====
• # 6. MULAI ANALISIS DATA DENGAN SPARK
• # =====
•
• print("📊 ANALISIS 1: Rata-rata Suhu & Kelembapan Berdasarkan Status Kamar")
• analisis_1 = df_spark.groupBy("status").agg(
•     count("*").alias("Total_Data"),
•     round(avg("suhu"), 2).alias("Rata_Suhu"),
•     round(avg("kelembapan"), 2).alias("Rata_Kelembapan")
• )
• analisis_1.show()

```

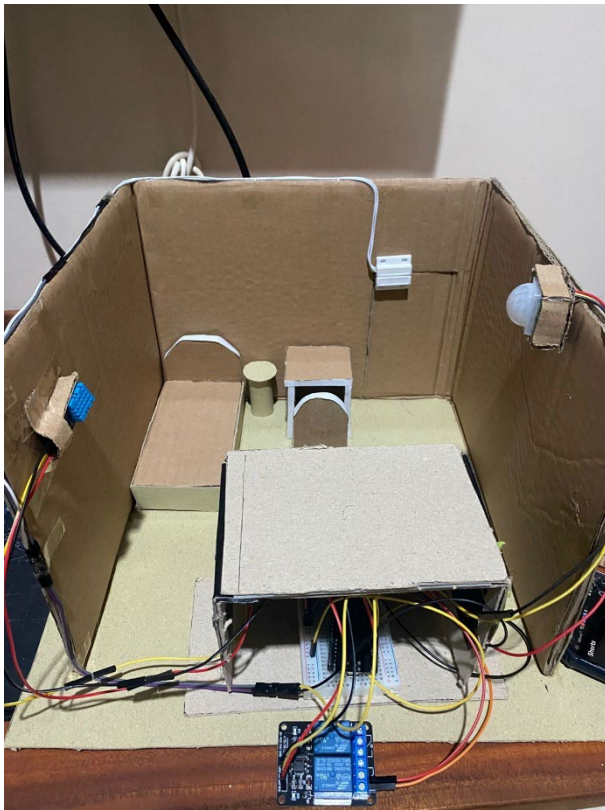
```

• print("📊 ANALISIS 2: Statistik Ekstrem (Suhu Tertinggi & Terendah)")
• analisis_2 = df_spark.select(
•     max("suhu").alias("Suhu_Maksimal"),
•     min("suhu").alias("Suhu_Minimal"),
•     max("kelembapan").alias("Kelembapan_Maksimal"),
•     min("kelembapan").alias("Kelembapan_Minimal")
• )
• analisis_2.show()
•
• print("📊 ANALISIS 3: Deteksi Aktivitas Pintu")
• analisis_3 = df_spark.groupBy("pintu").agg(
•     count("*").alias("Frekuensi_Kejadian")
• )
• analisis_3.show()
•
• # Selesai
• spark.stop()
• print("Selesai menganalisis data!")

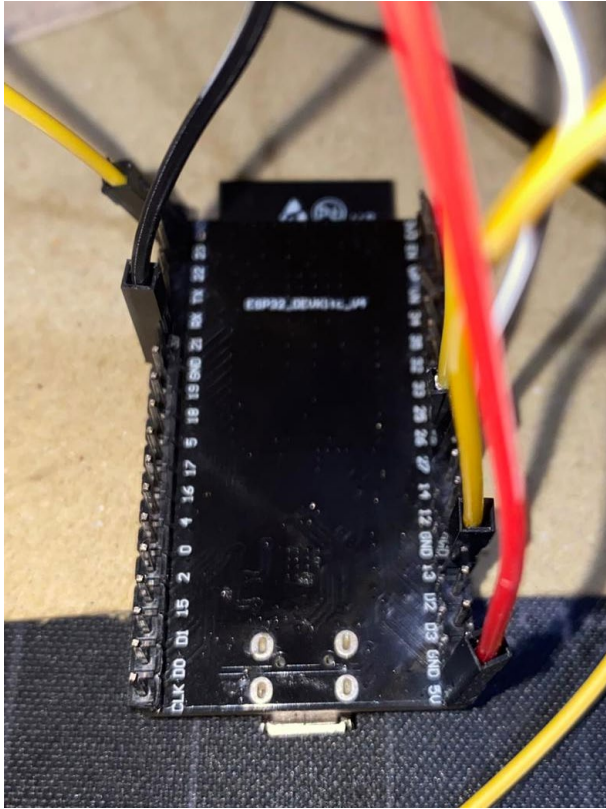
```

B. Dokumentasi Perangkat Keras

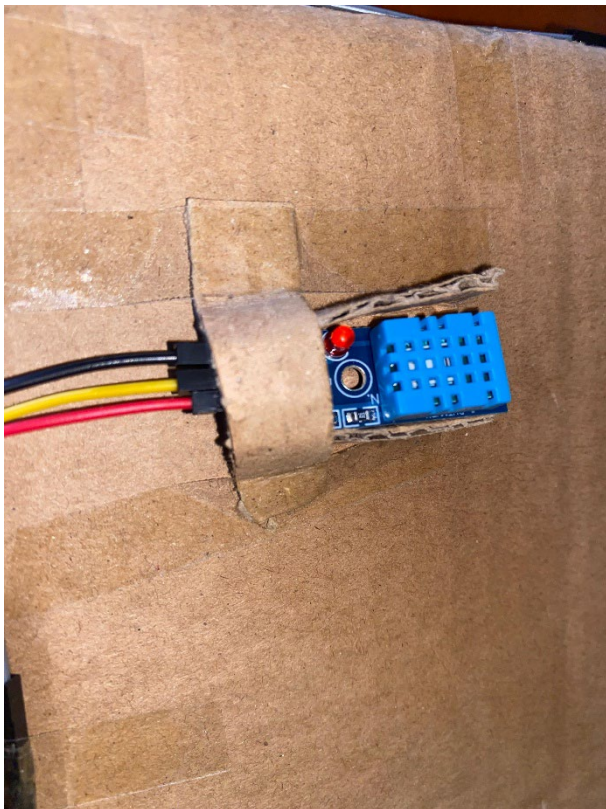
- Keseluruhan



- ESP32



- Sensor DHT11



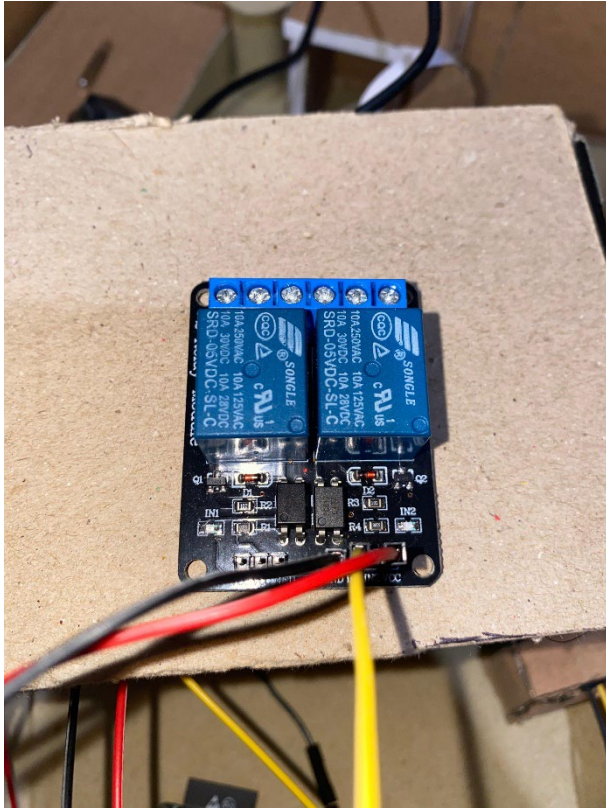
- Sensor PIR



- Sensor Magnetic Switch



- Relay



C. Tampilan Antarmuka

- Grafana



- Apache Spark

Machines - Tailscale

Spark Master at spark://127.0.0.1:7077

127.0.0.1:8080

Spark

4.1.1

Spark Master at spark://127.0.0.1:7077

URL: spark://127.0.0.1:7077

REST URL: spark://127.0.0.1:6066 (cluster mode)

Workers: 1 Alive, 0 Dead, 0 Decommissioned, 0 Unknown

Cores in use: 16 Total, 2 Used

Memory in use: 12.8 GiB Total, 1024.0 MiB Used

Resources in use:

Applications: 1 Running, 2 Completed

Drivers: 0 Running (0 Waiting), 0 Completed (0 Killed, 0 Failed, 0 Error, 0 Relaunching)

Status: ALIVE (Environment, Log)

Workers (1)

Worker Id	Address	State	Cores	Memory
worker-20260606153239-192.168.1.5-27758	192.168.1.5:27758	ALIVE	16 (2 Used)	12.8 GiB (1024.0 MiB Used)

Running Applications (1)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20260606162339-0002	(kill) AnalisisSmartHotel	2	1024.0 MiB		2026/06/06 16:23:39	Nestor	RUNNING	2 s

Completed Applications (2)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20260606153410-0001	AnalisisSmartHotel	2	1024.0 MiB		2026/06/06 15:34:10	Nestor	FINISHED	12 s
app-20260606153331-0000	AnalisisSmartHotel	2	1024.0 MiB		2026/06/06 15:33:31	Nestor	FINISHED	12 s

- Bot Telegram

11.27

5G 36

←

SM

Smart Hotel Bot

bot

⋮

📷 Set Profile Photo

June 7

/start 11:05 AM ✓✓

halo 11:05 AM ✓✓

TESTING: Sistem Smart Hotel mulai berjalan. Bot Telegram berhasil terhubung! 11:16 AM

🔧 INFO HOUSEKEEPING

Kamar 101 baru saja KOSONG.

Silakan bersihkan kamar! 11:27 AM

😊 Message

📎

📷